

015: StepPPO-v3 算法改动说明

2026-06-09

本文说明从 StepPPO-v2 到 StepPPO-v3 需要做的算法和工程改动。目标是先实现一个清晰、可诊断、低干扰的 v3-core，而不是一次性加入所有可能 trick。

1. v2 的问题边界

StepPPO-v2 的当前配置：

```
algorithm.step_ppo.step_advantage_w: 0.5
algorithm.step_ppo.episode_mode: mean_norm
algorithm.step_ppo.final_advantage_mode: direct
algorithm.step_ppo.normalize_episode_advantage: false
algorithm.step_ppo.normalize_step_advantage: true
algorithm.step_ppo.normalize_final_advantage: false
actor_rollout_ref.actor.value_head.detach_value_backbone: false
actor_rollout_ref.actor.value_head.value_loss_coef: 0.03
actor_rollout_ref.actor.ppo_micro_batch_size_per_gpu: 8
```

已有日志暴露四个问题：

1. `detach_value_backbone=false` 时，value loss 会回传 actor backbone；GiGPO value-aux 已经证明仅增加 auxiliary value regression 就可能破坏原 GiGPO。
2. StepPPO-v2 的 episode advantage 可能读取 raw `episode_rewards`，而 step reward 包含 invalid action penalty，导致 reward 口径不一致。
3. final advantage 同时混合 `A_epi` 和 `A_step`，但两者的尺度、归一化和 reward 语义不同。
4. policy loss 仍然是 token-level clipping，而 WebShop 的 environment action 是完整 response/action string。

v3-core 对应地做四个改动：

```
shared value backbone update  -> value-head-only update
raw episode reward mix        -> shaped step reward only
episode + step hybrid adv     -> pure step GAE advantage
token-level PPO ratio         -> action/sequence-level PPO ratio
```

2. 算法差异表

项目	StepPPO-v2	StepPPO-v3-core
MDP 粒度	step advantage + token PPO loss	environment step / response action
reward source	episode_rewards + step_rewards	shaped step reward only
invalid penalty	step term 可见, episode term 可能不可见	policy/value 全部可见
episode advantage	GiGPO-style mean_norm	删除
step advantage	GAE + batch whitening	GAE + uid group whitening
final advantage	A_epi + 0.5 * A_step	A_step_group_norm
value head	shared head, no detach	shared head, detach backbone
value loss	token-shaped/broadcast path	step scalar value loss
policy ratio	token-level PPO ratio	sequence geometric ratio
loss aggregation	token-oriented	seq-mean-token-mean
primary risk	value/backbone interference, reward mismatch	value underfit, lower update strength

3. New Config Namespace

建议新增独立 namespace, 避免继续复用 v2 的 `algorithm.step_ppo` 语义:

```

algorithm:
  adv_estimator: step_ppo_v3
  gamma: 0.95
  step_ppo_v3:
    reward_source: shaped_step
    use_episode_advantage: false
    step_gamma: 0.95
    step_lam: 0.90
    advantage_norm: uid
    zero_no_variance_group: true
    policy_ratio: sequence_geometric
    final_advantage_mode: pure_step
    min_group_std: 1.0e-6

actor_rollout_ref:
  actor:
    policy_loss: action_level
    loss_agg_mode: seq-mean-token-mean
    value_head:
      enable: true
      value_position: pre_action_last_context_token
      detach_value_backbone: true
      value_loss_coef: 0.03
      cliprange_value: 0.5

```

保留 `algorithm.gamma=0.95` 是为了和当前 GiGPO WebShop 口径对齐。`step_lam=0.90` 是 v3 的新默认值, 用于降低 step return 方差。

4. Advantage 计算改动

4.1 删除 episode score 输入

v2 逻辑中:

```
if "episode_rewards" in data.non_tensor_batch:
    episode_scores = data.non_tensor_batch["episode_rewards"]
else:
    episode_scores = token_level_rewards.sum(dim=-1)
```

v3 不再把 episode_scores 传入 policy advantage。episode_rewards 可以继续作为 logging 字段, 但不能进入 A_v3。

4.2 统一 shaped step reward

新增 helper:

```
def build_shaped_step_rewards(data, config) -> torch.Tensor:
    rewards = raw_rewards
    rewards -= invalid_action_penalty_coef * (1 - is_action_valid)
    if use_kl_in_reward:
        rewards += (token_level_rewards - token_level_scores).sum(dim=-1)
    return rewards
```

该 helper 应同时服务:

```
step GAE advantage
step return target
reward diagnostics
```

不要再出现 policy advantage 使用 raw reward、value target 使用 penalized reward 的分裂。

4.3 新增 v3 GAE 输出

建议新增函数:

```
def compute_step_ppo_v3_advantage_return(
    step_rewards: torch.Tensor,
    state_values: torch.Tensor,
    uid: np.ndarray,
    traj_index: np.ndarray,
    step_index: np.ndarray,
    gamma: float,
    lam: float,
    sample_index: np.ndarray | None,
    norm: str = "uid",
    min_group_std: float = 1e-6,
):
    ...
    return advantages, step_returns, diagnostics
```

输出:

```

advantages: shape [num_step_samples], scalar per environment action
step_returns: shape [num_step_samples], scalar per environment action
diagnostics:
    raw_adv_mean/std
    norm_adv_mean/std
    no_variance_group_ratio

```

注意: `compute_step_gae_advantage_return` 当前已经处理了 `step_sample_uid` 去重逻辑, v3 应复用这部分能力, 不要重新引入 duplicated row 被当作 fake transition 的问题。

4.4 uid group whitening

新增 normalization:

```

for each uid group:
    group_adv = raw_adv[group_indices]
    if group_adv.numel() <= 1 or group_adv.std() < min_group_std:
        normalized[group_indices] = 0
        no_variance_group_count += 1
    else:
        normalized[group_indices] = (group_adv - group_adv.mean()) / (group_adv.std() + eps)

```

理由:

- 避免不同 task difficulty 混在一个 batch 里互相缩放。
- 保留 GiGPO/GRPO 的 group-relative 稳定性。
- 对全成功或全失败、没有相对差异的 group 自动降权。

5. Policy Loss 改动

5.1 v2 token-level PPO

v2 当前 token-level ratio:

```

ratio = exp(log_prob - old_log_prob)
pg_loss = min(ratio * A_token, clip(ratio) * A_token)

```

同一个 response 内每个 token 单独 ratio、单独 clipping。

5.2 v3 sequence geometric ratio

v3-core 使用 response/action-level ratio:

```

token_log_ratio = (log_prob - old_log_prob) * response_mask
seq_len = response_mask.sum(dim=-1).clamp(min=1)
seq_log_ratio = token_log_ratio.sum(dim=-1) / seq_len
seq_ratio = exp(seq_log_ratio)

```

然后:

```
pg_loss_i = -min(
    seq_ratio_i * A_i,
    clip(seq_ratio_i, 1 - eps_low, 1 + eps_high) * A_i,
)
```

实现选择：

1. 最小改动：复用已有 `compute_policy_loss_gspo`，并设置 `actor.policy_loss=gspo` 或新增 alias `action_level`。
2. 更清晰改动：新增 `compute_policy_loss_action_level`，返回 action-level metrics，不复用 GSPO 命名。

建议先走第 2 条，避免 method 文档和代码里混用 GSPO 名称。

5.3 必加 action-level metrics

新增：

```
actor/action_log_ratio_mean
actor/action_log_ratio_std
actor/action_ratio_mean
actor/action_ratio_std
actor/action_clipfrac
actor/action_approx_kl
```

token-level metrics 可以保留，但不能只看 token-level `ppo_kl`。v3 要判断的是完整 response/action 是否被 PPO trust region 约束住。

6. Value Loss 改动

6.1 detach backbone 默认开启

v3 script 默认：

```
actor_rollout_ref.actor.value_head.detach_value_backbone: true
```

代码已经支持该字段，`value_head.py` 中会在 value head 前 detach hidden states。因此 v3 不需要先改 value head forward。

6.2 scalar value loss

当前 `step_returns` 在实现里容易沿 response token 维度 broadcast。v3 建议改成 step scalar：

```
current_values: [batch]
old_values: [batch]
step_returns: [batch]
```

loss：

```

values_clipped = old_values + clamp(current_values - old_values, -cliprange_value,
    cliprange_value)
value_loss_unclipped = (current_values - step_returns).pow(2)
value_loss_clipped = (values_clipped - step_returns).pow(2)
value_loss = 0.5 * max(value_loss_unclipped, value_loss_clipped).mean()

```

这和现有 StepWisePPOActor 的核心公式一致，但 v3 要确保 value loss 是 per action/step，而不是按 response token 重复加权。

6.3 value diagnostics

新增：

```

value_error = current_values.detach() - step_returns
value_rmse = sqrt(mean(value_error ** 2))
value_mae = mean(abs(value_error))
explained_variance = 1 - var(step_returns - pred) / (var(step_returns) + eps)

```

并记录分布：

```

actor/value_pred_std
actor/value_return_std
actor/value_return_p05/p50/p95

```

invalid action 分层：

```

actor/value_loss_valid
actor/value_loss_invalid
critic/adv_valid_mean
critic/adv_invalid_mean

```

7. DataProto 字段约定

v3 建议保持字段清晰：

```

data.batch["advantages"]      token-shaped or step-shaped policy advantages
data.batch["step_advantages"] scalar [batch] final v3 step advantage
data.batch["step_returns"]    scalar [batch] value target
data.batch["value_step_rewards"] scalar [batch] shaped step rewards
data.batch["state_values"]    scalar [batch] old state values

```

如果 actor update path 仍要求 token-shaped advantages，可以在最后一步 broadcast：

```

advantages_token = step_advantages.unsqueeze(-1) * response_mask

```

但 step_advantages 必须保留为 scalar 字段，用于 action-level loss 和 diagnostics。

8. 需要修改的文件

8.1 verl/trainer/ppo/step_ppo_algos.py

新增：

```

build_shaped_step_rewards
normalize_advantage_by_uid
compute_step_ppo_v3_advantage_return
compute_step_ppo_v3_advantage_data

```

或在现有 `compute_step_ppo_advantage_data` 中通过 `algorithm.adv_estimator == "step_ppo_v3"` dispatch。

8.2 verl/trainer/ppo/core_algos.py

新增:

```
compute_policy_loss_action_level
```

该函数可以从 `compute_policy_loss_gspo` 拆出来, 但需要返回 action-level metrics。

8.3 verl/workers/actor/step_ppo_actor.py

改动:

```

actor.policy_loss == "action_level"
scalar step_advantages / step_returns
action-level loss
action-level ratio metrics
value RMSE/MAE/EV/quantile metrics

```

8.4 verl/trainer/config/step_ppo_trainer.yaml

新增:

```

algorithm:
  step_ppo_v3:
    reward_source: shaped_step
    use_episode_advantage: false
    step_gamma: 0.95
    step_lam: 0.90
    advantage_norm: uid
    zero_no_variance_group: true
    min_group_std: 1.0e-6
    policy_ratio: sequence_geometric
    final_advantage_mode: pure_step

```

actor config 增加:

```
policy_loss: token_level # choices: token_level, gspo, action_level
```

8.5 EXPS/run_webshop_step_ppo_v3_4gpu_paper_align_simple.sh

新增主实验脚本, 建议基于 v2 脚本改:

```

bash "$BASE_SCRIPT" vllm \
  algorithm.adv_estimator=step_ppo_v3 \
  algorithm.gamma=0.95 \
  algorithm.step_ppo_v3.step_gamma=0.95 \
  algorithm.step_ppo_v3.step_lam=0.90 \
  algorithm.step_ppo_v3.advantage_norm=uid \
  algorithm.step_ppo_v3.use_episode_advantage=false \
  actor_rollout_ref.actor.policy_loss=action_level \
  actor_rollout_ref.actor.loss_agg_mode=seq-mean-token-mean \
  actor_rollout_ref.actor.value_head.detach_value_backbone=true \
  actor_rollout_ref.actor.value_head.value_loss_coef=0.03 \
  actor_rollout_ref.actor.ppo_micro_batch_size_per_gpu=8 \
  trainer.experiment_name=step_ppo_v3_qwen2.5_1.5b_4gpu_paper_align_full_e250 \
  "$@"

```

9. Smoke Test Checklist

2GPU smoke 必须先检查:

1.	actor/action_ratio_mean	actor/action_clipfrac		
2.	actor/value_rmse	actor/value_explained_variance	NaN	
3.	actor/value_loss	detach=true	actor grad norm	value loss
4.	episode/valid_action_ratio		0.5	
5.	response_length/clip_ratio		v2	
6.	no_variance_group_ratio		1	uid group norm policy signal

如果 smoke 中出现 action_ratio_mean 爆炸:

sequence geometric ratio	raw exp(sum log-ratio)
response_mask	padding
loss_agg_mode	seq-mean-token-mean

如果 value metrics 全部正常但 valid action ratio 仍低:

reward parser / invalid penalty	shaped step reward
v3_no_action_ratio	action-level ratio behavior

10. Ablation Matrix

首轮不要铺太多实验, 建议:

实验	改动	目的
v3_core	默认 v3	主线
v3_no_action_ratio	policy_loss=token_level	判断 action-level clipping 是否必要
v3_detach_false	detach_value_backbone=false	复验 shared-backbone interference
v3_lam_1	step_lam=1.0	判断高方差是否来自 MC target
v3_batch_norm_adv	advantage_norm=batch	判断 uid norm 是否必要

对照必须包含:


```

GiGP0 seed 2026
StepPPO-v2 seed 2026
GiGP0 value-aux detach=true seed 2026

```

11. 成功/失败判据

11.1 稳定性判据

v3_core 最低稳定性标准：

```

valid_action_ratio >= 0.85 after warmup
response_clip_ratio close to GiGP0 order of magnitude
action_clipfrac not persistently > 0.3
value_loss no repeated spikes above 8
no validation collapse similar to value-aux step 180--185

```

11.2 效果判据

如果 v3_core step 200 仍显著低于 StepPPO-v2，先不要继续加复杂 critic，优先定位：

```

reward
step_lam          credit
uid whitening
action-level ratio

```

如果 v3_core 稳定但只接近 StepPPO-v2：

```

step_lam=0.95
value_loss_coef=0.01/0.05
separate value head lr
GAGPO-style grouped value proxy    learned value target

```

12. 不纳入 v3-core 的内容

以下内容不进入 v3-core，避免主线过宽：

1. separate critic model。
2. value loss 回传 backbone。
3. in-distribution critic pretraining。
4. Huber value loss 默认开启。
5. PRM / external reward model。
6. GAGPO grouped value proxy 替代 learned value。
7. dynamic task sampling。

这些方向都可能有价值，但应在 v3-core 稳定后作为 v3.1/v4 单独验证。

13. Implementation Order

建议实现顺序:

1. 新增 v3 advantage 计算和 shaped reward helper。
2. 新增 action-level policy loss 和 metrics。
3. 改 StepWisePPOActor 支持 scalar `step_advantages` / `step_returns`。
4. 增加 value diagnostics。
5. 增加 config namespace 和 v3 run script。
6. 跑 2GPU smoke。
7. 跑 seed 2026 full 250 step。
8. 对比 GiGPO、StepPPO-v2、GiGPO value-aux detach=true。

每一步完成后都应保留可回退的独立开关，避免 v3 出问题时只能整体回滚。